What is the problem?
Food from a household is being consumed way sooner than expected with no idea on who is consuming what and why is the food lasting so little when it's bought to last a whole month. To have better control over what is going on, a family needs to implement an app that will have a log of the grocery items bought during shopping, knowing how much was bought from each item, when it was bought and the date it was fully consumed. A dashboard with insights about the items bought is also expected.

What is the work to be done?
Implement a cloud-hosted web application where each one of the house-hold members can access the portal to add a new grocery shopping with different items and information about them, to remove food from the log once it has been consumed, or view who has used the app and implemented changes. The data we'll have to log the food will be simple enough, but the ERD and the whole database structure still needs to be created from scratch with a rough idea of how the tables will look like and the relationship among the permits each user has, the user, and the food logs. The schemes and ideas of the database will be
FoodLog EDR scheme - Draft
The web application will have 3 views.The different views of the application will be described as it follows:

1. The first view will be the Login / Register view. One can access through the Google account. For the time being this will be the only form of accessing the tool.
2. The main grocery food log: This view will have the logs of the different grocery shoppings made. Each entry on this log will be grocery shopping. Think of it as logging the whole bag of groceries. One bag can have several items but one item has to belong to one bag. For each grocery shopping done, the relevant columns will be:
    a. groceries_shopping_id: Unique distinct id of going to buy something
    b. entry_date: The date in which the food was bought
    c. store_name: The name of the store where the food was bought.

    For each of the views, there should be a bar showing the process of food consumption as well. If for example the total of items bought in that shopping was 20, and they have fully consumed 10, then the bar should be half full and showing a 50% consumption. As this view will have each of the grocery shopping logged, the information on this grocery shopping should appear, such as the information above and some new calculated values as:
    a. The number of items bought: each unique distinct item bought.
    b. The total price of the grocery shopping: how much was the total. This number can be calculated by adding the individual price of each item that belongs to a grocery shopping.
    c. The number of items fully consumed belonging to that grocery shopping: Can be calculated by counting the number of items with the consumed_date non null from the specific grocery_shopping_id.
    d. Total days that have passed from that grocery shopping: Calculated as the day in which the food was bought minus the actual day.

    In this view, there will be a dashboard at the start with some main statistics like:

a. Number of grocery shoppings made: Calculated as the number of unique distinct grocery_shopping_id
b. Last date in which a grocery_shopping_id was created with 10 or more items
c. Number of items not yet fully consumed from all of the different grocery_shopping_id. In other words, all of the food items that have a consumed_date as null.
d. Total amount of money spent of grocery shopping in a month (full 30 days)

After that, there will be some filters such as:

a. Price: this will have an open field to put the price and a button to specify if the amount on the previous field is greater than, greater of equal than, smaller than, of smaller or equal than that amount
b. Store name: A dropdown menu with all of the store names in the database.
c. Food consumption: A dropdown menu to choose what to show: all the grocery shoppings that have both consumed and not fully consumed yet, all of the shoppings that have fully consumed items, and the shoppings that have yet still items not fully consumed.

After those filters, there should be a button to add a new grocery shopping list. If you press the button, this will take you to a subpage of that grocery shopping were all of the information will be filled, meaning there will be a unique distinct ID created for that grocery shopping, an entry date whose default value will be “today” (or whenever the app is being used) with an option to select another date, and a field with a dropdown menu to choose from the previous store names or place a new name if the dropdown menu doesn’t show any of the correct options.

Below that, there will be an “empty table” with 5 rows, and below those 5 rows a button to add more rows and to the farthest right end of the row, an option to delete that row. This empty table is going to be filled with the information of each individual item from the grocery shopping. The information of the items belonging to that grocery shopping should look like this:

a. Id: Item unique id (this should be automatically created by the app, not by the human and non visible to that person)
b. A row number for the grocery list.
c. Name: The name of the food being added
d. Brand: brand of the name of the food being added
e. Size: size of the item bought. Ej. 200mL of Coke, 1000g of ground beef. The units here will be mililiters or grams.
f. Price: price of the item when it was bought. The currency here will be COP (Colombian peso)
g. Store_bought: Name of the store where the item was bought at. This information should come from the grocery_shopping_id
h. Entry_date: Date the food enters the log. The default option while being added will be today’s date (whenever today happens) but it should be able to edit the date to whenever I want. For example: if the item was bought 2 days ago, I should be able to put the date of 2 days ago although the default would show the “today’s” date. This will be the same date as the grocery_shopping_id

i. Consumed_date: The date at which the item had been fully consumed. This field should be empty until the user hits the “Fully consumed” button. On a later stage of the process.
j. Notes: a little text box with notes to write whatever the user wants on the item.

The fields being filled by the user while adding the information should be: name, brand, size, and price. The other fields of information will be shown when the user wants to inspect a list and view the items bought. This will happen once the user finishes adding the items to the list and clicking on a confirmation button that the list is complete. A pop-up message will require the user to reconfirm that the list is complete. There will also be an option to save as a draft to finish the list later.

To view the draft list, there will be an option on the main good log menu right beside the add new grocery list. By clicking this button, a subpage will emerge with all the grocery shoppings drafts and by clicking one of those, the editable table with information should reappear. Once the draft is complete and no more items are going to be added, the list will not appear on the draft subpage.

As a part of the subpage while viewing a grocery shopping that has been logged, the items will have all of the fields mentioned before and right next to the notes column, there will be a button saying “Fully consumed”. This one will mark the date at the column “consumed_date” when it's pressed. When being pressed, there will be a pop-up message saying to confirm that the item “name of the item” has been consumed and a button in that pop-up to confirm and another one to cancel. There will also be a second button under that one that will say “Delete”, which will show a pop-up forcing the user to type ‘DELETE’ so the log can be removed. Similar to the way GitHub forces someone to type something before deleting the item. The pop-up also needs the option to go back if needed. This option will only be available to those whose role is “admin”.

The full grocery view also needs pagination to bring 20 columns at a time, just like Gmail brings 50 emails at a time.

3. Insight dashboard: Still in development. Do nothing here for the time being.
4. Users / roles / logs: A view divided in two:
    a. A “Users and permissions sets”, which will be a table having the User(Name and email used), the role that person has (adminis can change the role for a person), the last login for that user, a counter with the number of views from each person, and the number of edits (add or delete food)
    b. A “Recent Activity” box, which contains the log of all of the different things a user has done on the page. That page will have the following columns: When (the activity happened), User (the user that made the action), Action (The action done; view, login, add food, complete food), Path / Resource (which page did the action occurred at), Change (if something changed such as adding or deleting food. This will log each of the changes done including if a Role changed for one of the users). This part of the webpage in will need to have a pagination, calling just 20 rows at a time and having a counter at the button saying which changes we are seeing. For example: Pages 21-40 of 320.

How are we going to implement it?

- It needs to be hosted in Vercel
- The database will happen in Supabase
- Everything will be uploaded to a repository on Github
- The main code for the database needs to be created using SQL
- The main code for the rest of the webpage will be developed using TypeScript
- The code needs to be optimized for phone view and usage, as this will be the main device from which food will be logged as new or fully consumed